Library Management System

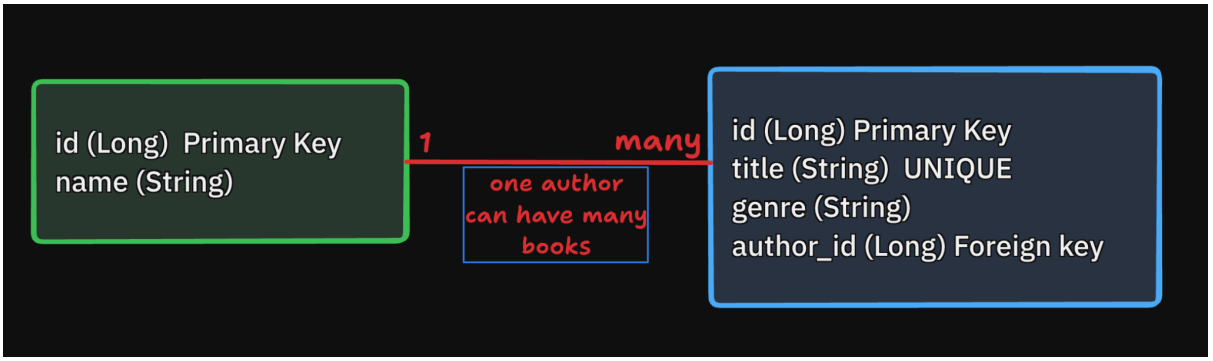
Project approach report covering ER design, operation implementation, code snippets, screenshots, challenges and testing.

1. Project Overview

This project is a Spring Boot MVC web application for maintaining a simple book catalog. It uses Spring Data JPA with Hibernate for persistence, an H-2 in memory database for development, JSP views for the user interface, and Maven for building and testing.

- **Backend:** Spring Boot 4.0.6, Spring MVC, Spring Data JPA, Hibernate
- **Database:** H2 in-memory database at `jdbc: h2: mem : librarydb`
- **Frontend:** JSP files under `src/main/ webapp/WEB-INF/jsp` with embedded CSS.
- **Main feature set:** list books with authors, add a book, edit an existing book, and show a clear error for duplicate or empty book titles.

2. Entity Relationship Design:



3. Implementation Details For each operation:

3.1 Read Operation: List books with authors

The list page is served by `GET /books`. The controller asks the service for all books, the service delegates to a custom repository query, and the JSP renders each book with its nested author name.

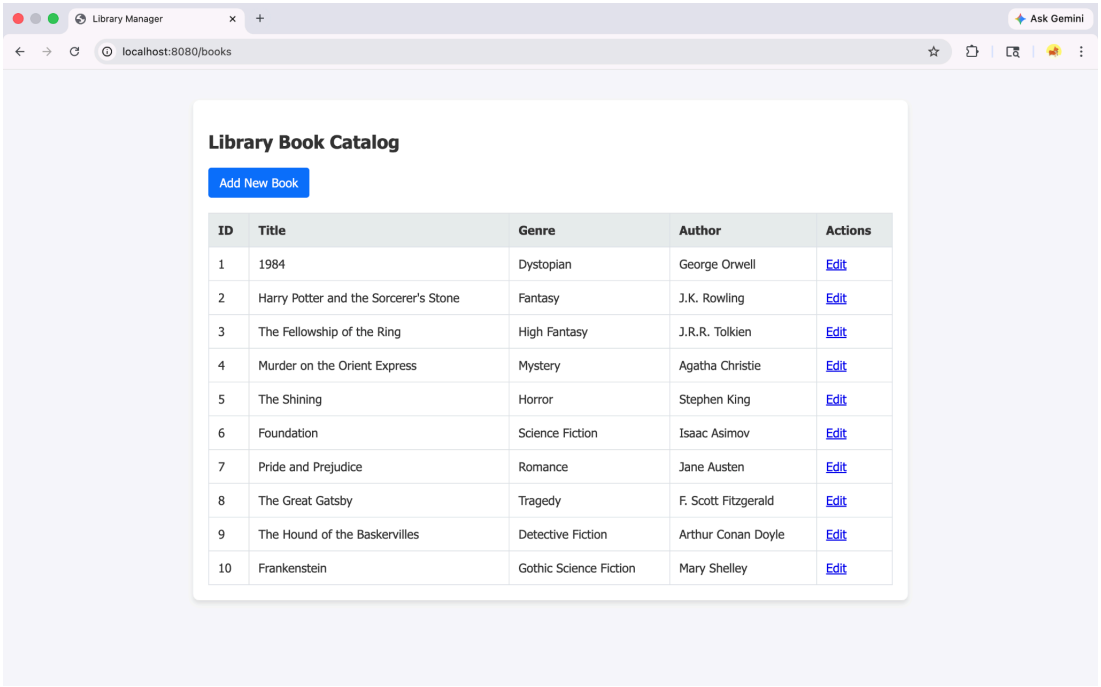
`Controller + service + repository flow`

```
// ── READ Operation ──
// Maps to GET requests at "/books"
@GetMapping
public String listBooks(Model model) {
    // Fetch all books (with authors) and add them to the Model.
    // The Model is how we pass data from Java to the JSP file.
    model.addAttribute("books", libraryService.getAllBooksWithAuthors());

    // Returns the name of the JSP file to render (list-books.jsp)
    return "list-books";
}
```

```
public List<Book> getAllBooksWithAuthors() {
    // Calling our highly optimized custom inner join query!
    return bookRepository.findAllBooksWithAuthorsInnerJoin();
}
```

```
@Query("SELECT b FROM Book b INNER JOIN FETCH b.author")
List<Book> findAllBooksWithAuthorsInnerJoin();
```



Book catalog rendered from seeded H2 data using the inner join query.

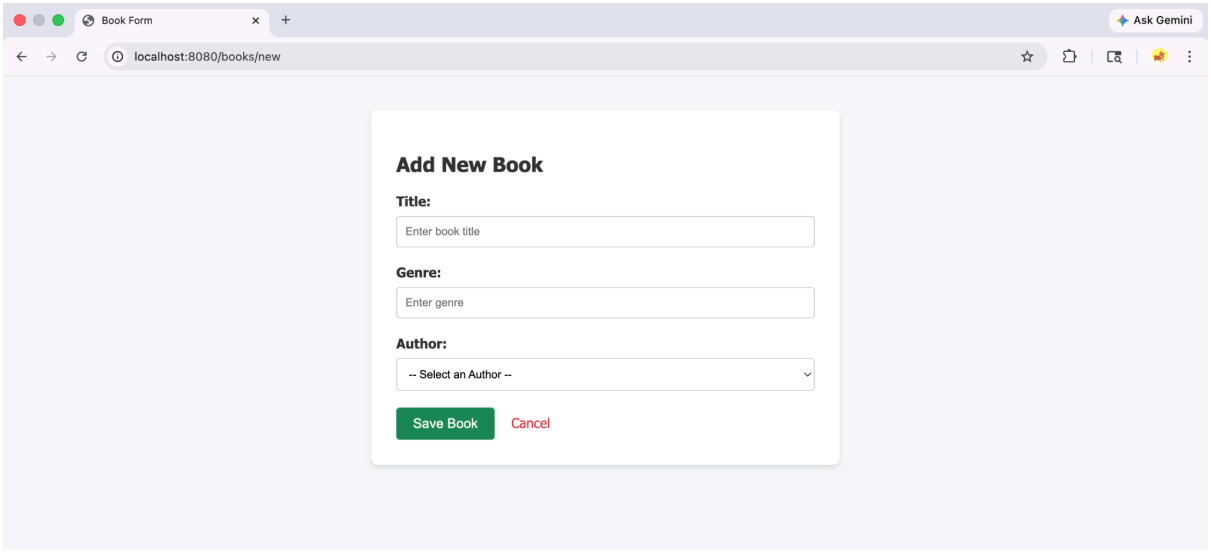
3.2 Create Operation: Show Add Book Form

The add form is served by `GET /books/new`. The controller sends an empty `Book` object for Spring form binding and all authors for the author dropdown.

`Controller code for opening the create form`

```
// ── CREATE Operation (Show Form) ──
// Maps to GET requests at "/books/new"
@GetMapping("/new")
public String showAddForm(Model model) {
    // We pass an empty Book object to the form so the JSP can bind inputs to it
    model.addAttribute("book", new Book());
    // We also need the list of authors so the user can select one from a dropdown
    model.addAttribute("authors", libraryService.getAllAuthors());

    return "book-form"; // Returns book-form.jsp
}
```



Add new book form with title, genre, and author dropdown

3.3 Save Operation: Insert a New book

The same POST /books/save endpoint handles saving. When the submitted book has no id, Spring Data JPA treats bookRepository.save(book) as an insert. A flash message is used after redirection so the user sees confirmation on the list page.

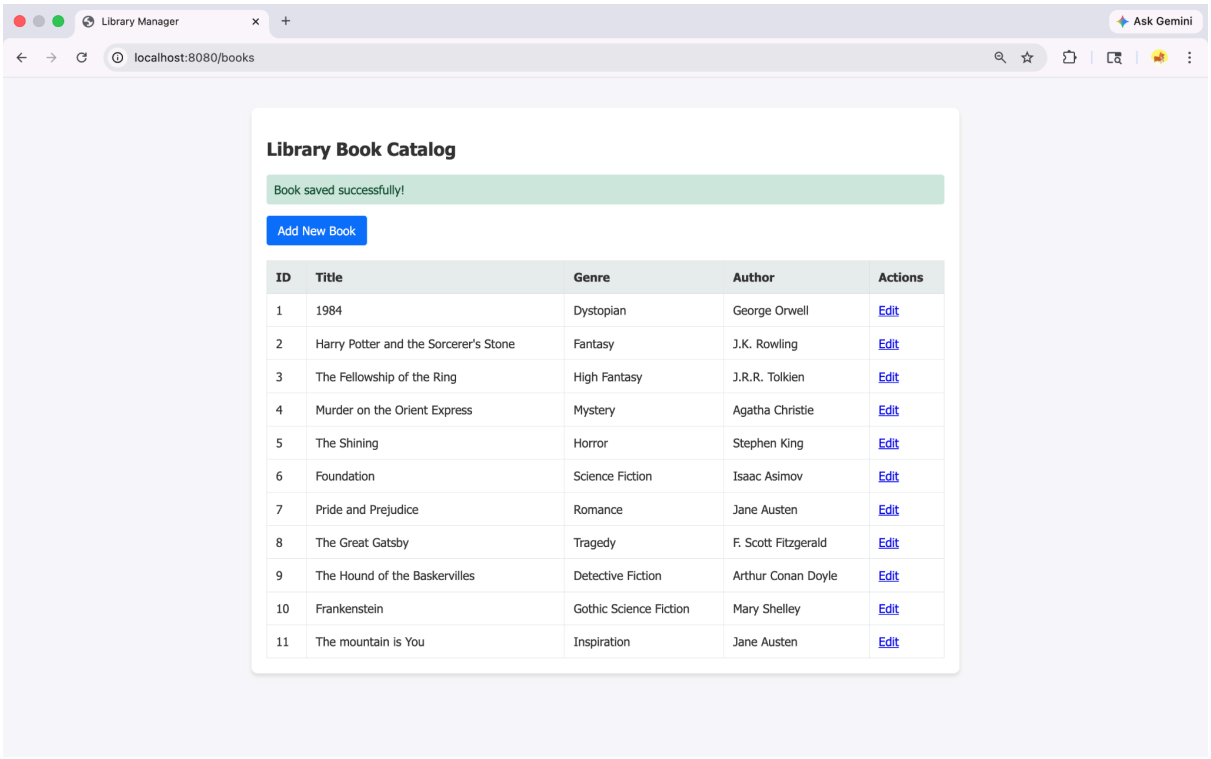
POST handler and service save method

```
// ── CREATE / UPDATE Operation (Process Form) ──
// Maps to POST requests at "/books/save"
@PostMapping("/save")
public String saveBook(
    @ModelAttribute("book") Book book,
    RedirectAttributes redirectAttributes
) {
    try {
        // Send the data to the Service layer to be saved
        libraryService.saveBook(book);
        // Flash attributes survive a redirect, perfect for success messages
        redirectAttributes.addFlashAttribute(
            "success",
            "Book saved successfully!"
        );
    } catch (RuntimeException e) {
        // If the Service layer throws our DataIntegrityViolationException, catch it here
        redirectAttributes.addFlashAttribute("error", e.getMessage());
        // Send them back to the form if there was an error
        return "redirect:/books/new";
    }

    // If successful, redirect back to the main list
    return "redirect:/books";
}
```

```java
// ── CREATE / UPDATE Operation ──

public void saveBook(Book book) {
    try {
        // If the book has no ID, save() creates a new record.
        // If the book HAS an ID, save() updates the existing record.
        bookRepository.save(book);
    } catch (DataIntegrityViolationException e) {
        // This catches the exact exception your rubric mentions!
        // It triggers if the title is null, or if it violates the unique constraint.
        throw new RuntimeException(
            "Data integrity violation: Book title must be unique and cannot be empty."
        );
    }
}
```



`Successful create operation redirects back to the catalog with a success message.`

3.4 Update Operation: Edit an existing Book

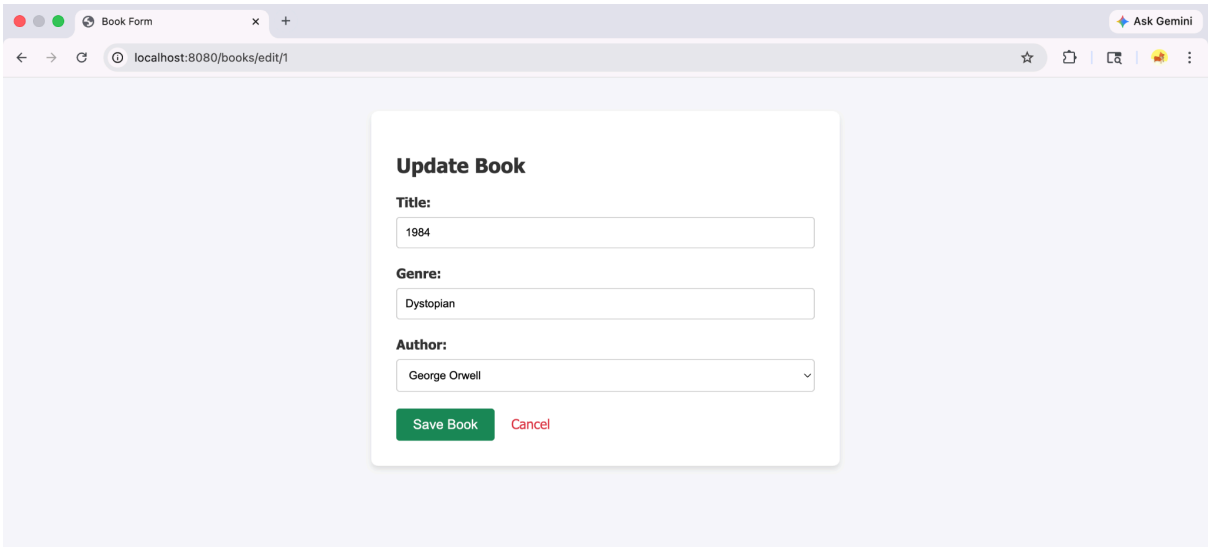
The edit form is served by `GET/books/edit{id}`. The controller loads the existing book and reuses the same JSP. Because the form includes a hidden id field, the save endpoint performs an update instead of an insert.

Controller code for opening the edit form

```java
// ── UPDATE Operation (Show Form with Data) ──
// Maps to GET requests at "/books/edit/{id}"
@GetMapping("/edit/{id}")
public String showEditForm(@PathVariable Long id, Model model) {
    // Fetch the existing book by its ID
    model.addAttribute("book", libraryService.getBookById(id));
    // Fetch authors for the dropdown
    model.addAttribute("authors", libraryService.getAllAuthors());

    // Notice we reuse the exact same form used for creating!
    return "book-form";
}
```

```java
public Book getBookById(Long id) {
    // .orElseThrow ensures we don't return a null object if the book isn't found
    return bookRepository
        .findById(id)
        .orElseThrow(() ->
            new RuntimeException("Book with ID " + id + " not found")
        );
}
```



The same form is reused for updating an existing book.

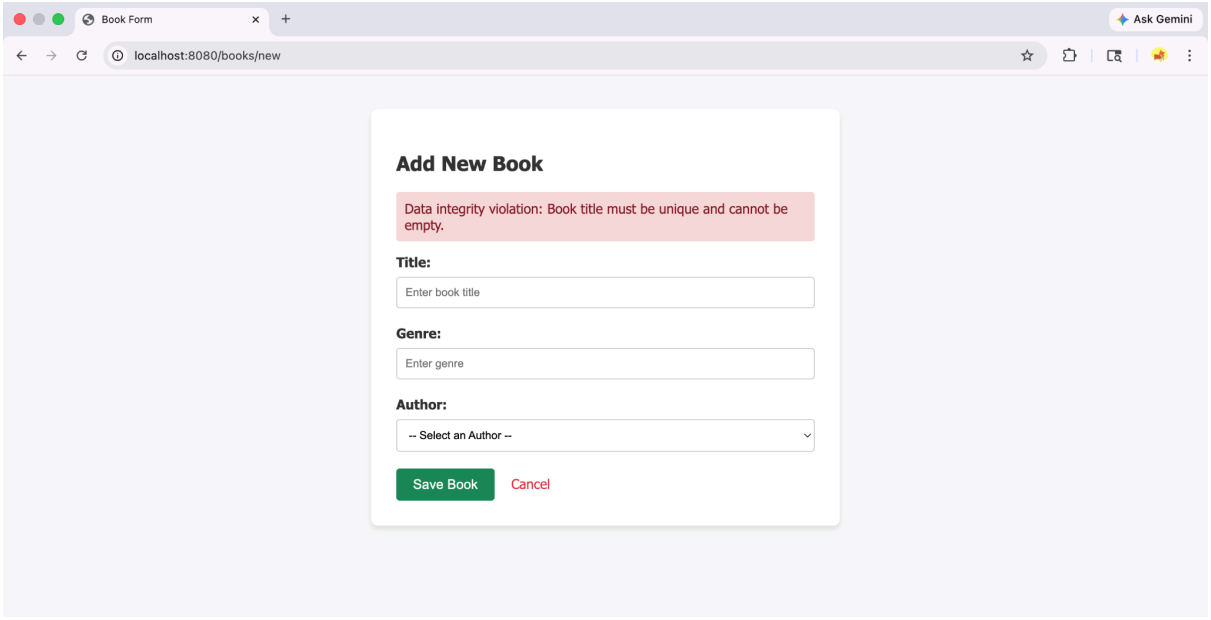
3.5 Data integrity handling

The Book.title column is both required and unique. If the database rejects a duplicate title, the service catches the Spring data exception and returns a friendly message to the form instead of exposing a raw SQL error.

Friendly integrity handling in LibraryService

```java
// ── CREATE / UPDATE Operation ──

public void saveBook(Book book) {
    try {
        // If the book has no ID, save() creates a new record.
        // If the book HAS an ID, save() updates the existing record.
        bookRepository.save(book);
    } catch (DataIntegrityViolationException e) {
        // This catches the exact exception your rubric mentions!
        // It triggers if the title is null, or if it violates the unique constraint.
        throw new RuntimeException(
            "Data integrity violation: Book title must be unique and cannot be empty."
        );
    }
}
```



Duplicate title submission is rejected and shown as a clear validation message.

4. Database Initialization

The application uses `spring.jpa.hibernate.ddl-auto=update` so Hibernate creates or updates the schema from the entity classes. The `data.sql` file seeds 10 authors and 10 books. `spring.jpa.defer-datasource-initialization=true` ensures the schema exists before seed data is inserted.

`Configuration: src/main/resources/application.properties`

```
# 1. Database Configuration (Telling Spring to use the H2 in-memory DB)
spring.datasource.url=jdbc:h2:mem:librarydb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

# Tell Hibernate to automatically create the tables based on our Java classes
spring.jpa.hibernate.ddl-auto=update

# Enable the H2 web console so we can look at our database later (Optional but helpful)
spring.h2.console.enabled=true

# 2. View Resolver Configuration (Telling Spring MVC where to find the UI)
spring.mvc.view.prefix=/WEB-INF/jsp/
spring.mvc.view.suffix=.jsp
# Tells Spring to wait until Hibernate creates the tables before running data.sql
spring.jpa.defer-datasource-initialization=true
```

`Seed data example: src/main/resources/data.sql`

```
INSERT INTO author (name) VALUES ('George Orwell');
INSERT INTO author (name) VALUES ('J.K. Rowling');
```

```
INSERT INTO book (title, genre, author_id) VALUES ('1984', 'Dystopian', 1);
INSERT INTO book (title, genre, author_id) VALUES ('Harry Potter and the Sorcerer''s Stone', 'Fantasy', 2);
```

5. Challenges Faced and How I overcame them:

- **Representing the author-book relationship cleanly:** Separated Author and Book into two JPA entities. Book has the foreign key with @ManyToOne and @JoinColumn, while Author exposes the inverse @OneToMany relationship.
- **Displaying author details without extra queries:** Used an explicit JPQL INNER JOIN FETCH query so the list page receives books and authors in one query.
- **Showing validation errors clearly:** Added a unique and not-null constraint on title, caught DataIntegrityViolationException in the service and displayed the error on the form.

6. Testing

The project includes Spring Boot tests for context loading and service behavior. The service tests verify that seeded books are returned, book id lookup works, authors are loaded, and duplicate titles produce the expected friendly error message.

```
@Test
void getAllBooksWithAuthorsReturnsSeededBooks() {
    List<Book> books = libraryService.getAllBooksWithAuthors();

    assertFalse(books.isEmpty());
    assertTrue(
        books.stream().anyMatch(book → "1984".equals(book.getTitle()))
    );
    assertTrue(books.stream().allMatch(book → book.getAuthor() ≠ null));
}
```

```
@Test
void saveBookThrowsFriendlyMessageOnIntegrityViolation() {
    Author author = libraryService.getAllAuthors().get(0);
    Book duplicate = new Book("1984", "Dystopian", author);

    RuntimeException exception = assertThrows(
        RuntimeException.class,
        () → libraryService.saveBook(duplicate)
    );

    assertEquals(
        "Data integrity violation: Book title must be unique and cannot be empty.",
        exception.getMessage()
    );
}
```

Verification: `./mvnw test`

```
➜  library ./mvnw test
[INFO] Scanning for projects...
[INFO]
[INFO] ------------------------< com.example:library >-------------------------
[INFO] Building  0.0.1-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
```

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.009 s -- in com.example.library.LibraryApplicationTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  3.605 s
[INFO] Finished at: 2026-05-02T14:24:34+05:30
[INFO] ------------------------------------------------------------------------
```

GitHub Repo Link:

https://github.com/yashranjit/BDA-SGA-2/